tf.while_loop Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/while_loop

> *Repeat body while the condition cond is true. (deprecated argument values)*

Function Signatures

```
tf.while_loop( cond, body, loop_vars, shape_invariants=None,
parallel_iterations=10, back_prop=True, swap_memory=False,
maximum_iterations=None, name=None )
(back_prop=False)
```

Code Examples

```
tf.while_loop(
cond,
body,
loop_vars,
shape_invariants=None,
parallel_iterations=10,
back_prop=True,
swap_memory=False,
maximum_iterations=None,
name=None
)
```

```
tf.while_loop(
cond,
body,
loop_vars,
shape_invariants=None,
parallel_iterations=10,
back_prop=True,
swap_memory=False,
maximum_iterations=None,
name=None
)
```

```
@tf.function
def sumSquare(n):
i, result = tf.constant(0), tf.constant(0)
while i < n: # AutoGraph converts while-loop to tf.while_loop().
result += i * i
i += 1
return result
sumSquare(10).numpy()
285
```

```
@tf.function
```

```
def sumSquare(n):
```

```
i, result = tf.constant(0), tf.constant(0)
```

```
while i < n: # AutoGraph converts while-loop to tf.while_loop().
```

```
result += i * i
```

Documentation

Repeat body while the condition cond is true. (deprecated argument values)

Used in the notebooks

• Extension types

- Modeling COVID-19 spread in Europe and the effect of interventions
- Linear Mixed Effects Models

For more information, see tf.function and AutoGraph guide
.

cond is a callable returning a boolean scalar tensor. body is a callable returning a (possibly nested) tuple, namedtuple or list of tensors of the same arity (length and structure) and types as loop_vars. loop_vars is a (possibly nested) tuple, namedtuple or list of tensors that is passed to both cond and body. cond and body both take as many arguments as there are loop_vars.

In addition to regular Tensors or IndexedSlices, the body may accept and return TensorArray objects. The flows of the TensorArray objects will be appropriately forwarded between loops and during gradient calculations.

Note that while_loop calls cond and body exactly once (inside the call to while_loop, and not at all during Session.run()). while_loop stitches together the graph fragments created during the cond and body calls with some additional graph nodes to create the graph flow that repeats body until cond returns false.

For correctness, tf.while_loop() strictly enforces shape invariants for
the loop variables. A shape invariant is a (possibly partial) shape that
is unchanged across the iterations of the loop. An error will be raised
if the shape of a loop variable after an iteration is determined to be more
general than or incompatible with its shape invariant. For example, a shape
of [11, None] is more general than a shape of [11, 17], and [11, 21] is
not compatible with [11, 17]. By default (if the argument shape_invariants
is not specified), it is assumed that the initial shape of each tensor in
loop_vars is the same in every iteration. The shape_invariants argument
allows the caller to specify a less specific shape invariant for each loop
variable, which is needed if the shape varies between iterations. The
tf.Tensor.set_shape
function may also be used in the body function to indicate that
the output loop variable has a particular shape. The shape invariant for
SparseTensor and IndexedSlices are treated specially as follows:

a) If a loop variable is a SparseTensor, the shape invariant must be
TensorShape([r]) where r is the rank of the dense tensor represented
by the sparse tensor. It means the shapes of the three tensors of the
SparseTensor are ([None], [None, r], [r]). NOTE: The shape invariant here
is the shape of the SparseTensor.dense_shape property. It must be the shape of
a vector.

b) If a loop variable is an IndexedSlices, the shape invariant must be
a shape invariant of the values tensor of the IndexedSlices. It means
the shapes of the three tensors of the IndexedSlices are (shape, [shape[0]],
[shape.ndims]).

while_loop implements non-strict semantics, enabling multiple iterations
to run in parallel. The maximum number of parallel iterations can be

controlled by parallel_iterations, which gives users some control over memory consumption and execution order. For correct programs, while_loop should return the same result for any parallel_iterations > 0.

For training, TensorFlow stores the tensors that are produced in the forward inference and are needed in back propagation. These tensors are a main source of memory consumption and often cause OOM errors when training on GPUs. When the flag swap_memory is true, we swap out these tensors from GPU to CPU. This for example allows us to train RNN models with very long sequences and large batches.

Returns

Raises

Example:

Example with nesting and a namedtuple:

Example using shape_invariants:

Example which demonstrates non-strict semantics: In the following example, the final value of counter does not depend on x. So the while_loop can increment the counter parallel to updates of x. However, because the loop counter at one loop iteration depends on the value at the previous iteration, the loop counter itself cannot be incremented in parallel. Hence if we just want the final value of the counter (which we print on the line print(sess.run(i))), then x will never be incremented, but the counter will be updated on a single thread. Conversely, if we want the value of the output (which we print on the line print(sess.run(out).shape)), then the counter may be

incremented on its own thread, while x can be incremented in parallel on a separate thread. In the extreme case, it is conceivable that the thread incrementing the counter runs until completion before x is incremented even a single time. The only thing that can never happen is that the thread updating x can never get ahead of the counter thread because the thread incrementing x depends on the value of the counter.

... # Updating i based on i == [9]
... # Updating x based on i == [3]
... # ... # meaning that the counter(i) thread is on iteration 9,
... # while the x thread is on iteration 3.
... print(sess.run(x_out).shape)
(1000, 100)